



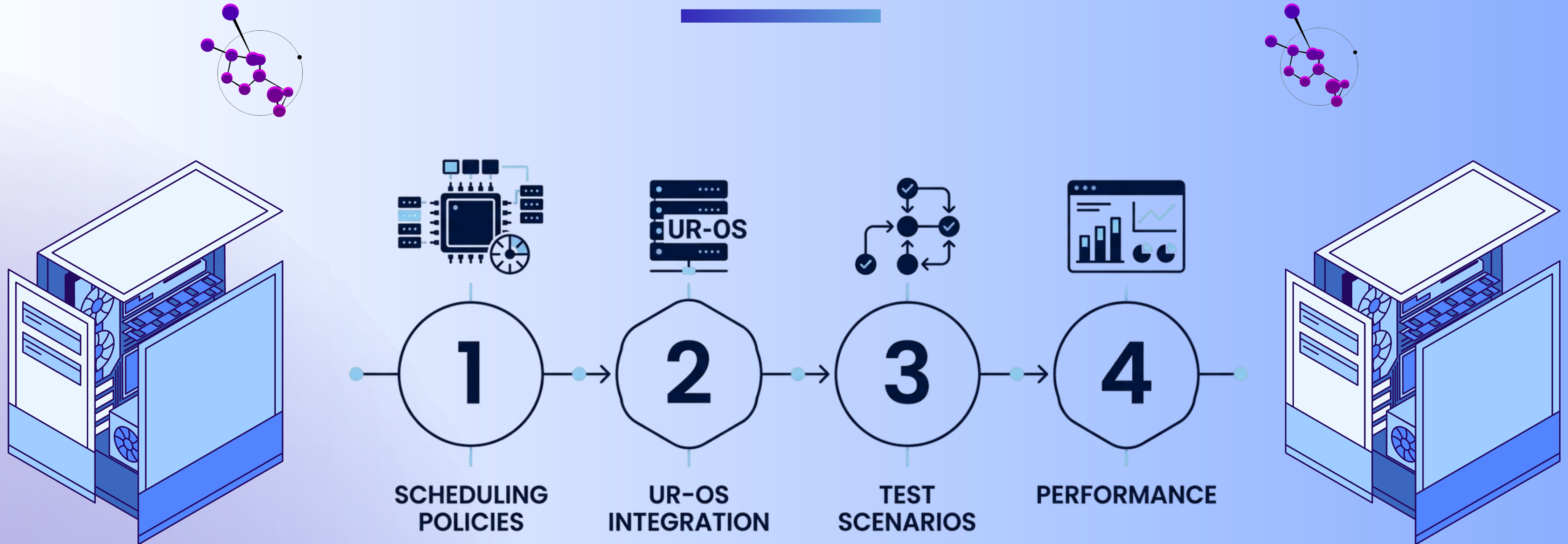
UR - OS

PROCESS SCHEDULING SIMULATOR

SEPTEMBER 2026



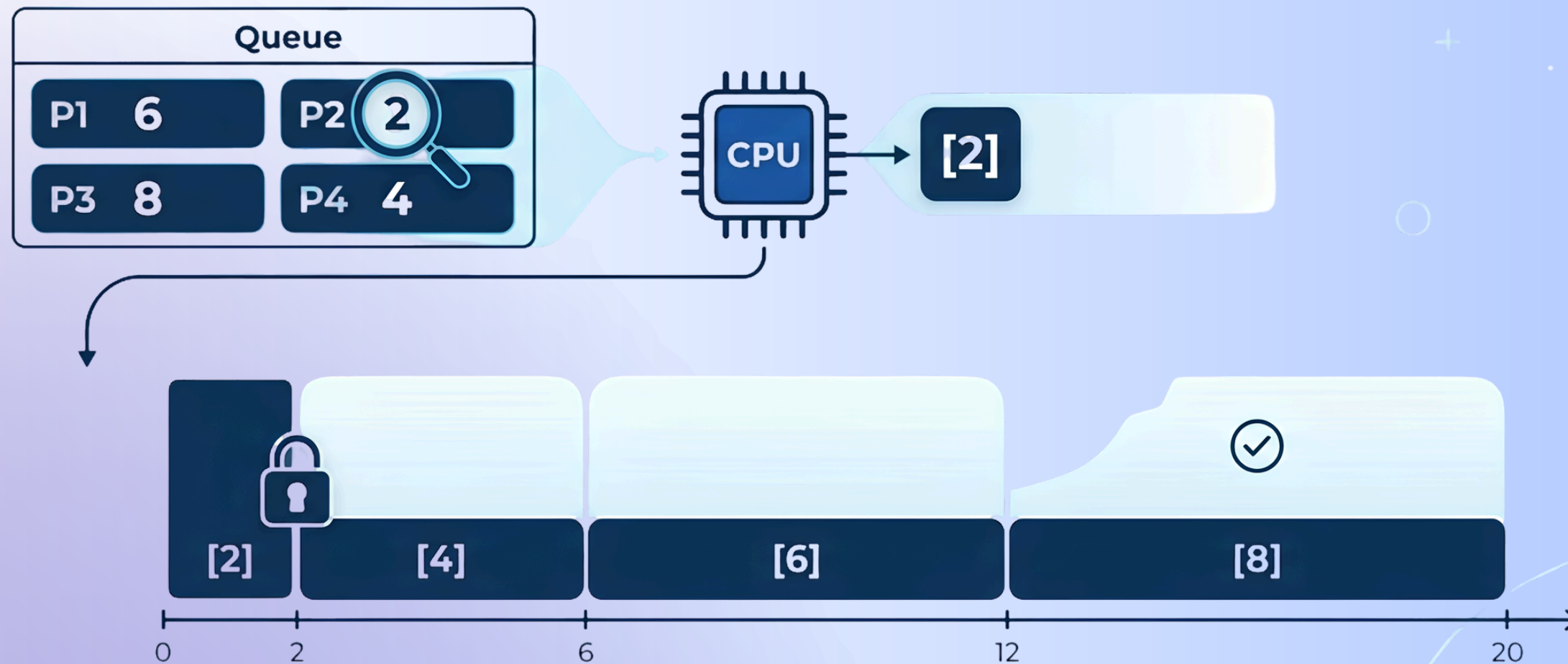
DEFINITION AND METOLOGY



SJF NON-PREEMPTIVE



 **SJF**



- Shortest First: The CPU picks the job with the shortest execution time from the queue.
- No Interruptions: Once a job starts running, it cannot be paused or stopped.
- Full Completion: The current job must finish its entire task to release the CPU.
- Next Job: The CPU selects the next shortest job only after the active one finishes completely.

IT IS USEFUL WHEN WE KNOW OR CAN ESTIMATE THE TIME REQUIRED FOR PROCESSES AND WANT TO REDUCE AVERAGE WAITING TIMES

SJF NON-PREEMPTIVE

```
public void getNext(boolean cpuEmpty) {  
    if (cpuEmpty && !processes.isEmpty()) {  
        Process shortest = processes.get(0);  
  
        // Find the process with the shortest remaining CPU time  
        for (Process p : processes) {  
            int pRemaining = p.getRemainingTimeInCurrentBurst();  
            int shortestRemaining = shortest.getRemainingTimeInCurrentBurst();  
  
            if (pRemaining < shortestRemaining  
                || (pRemaining == shortestRemaining && tieBreaker(p, shortest) == p)) {  
                shortest = p;  
            }  
        }  
  
        os.interrupt(InterruptType.SCHEDULER_RQ_TO_CPU, shortest);  
        processes.remove(shortest);  
    }  
}
```


SJF PREEMPTIVE



SJF PREEMPTIVE (SRTF)

- Shortest Remaining Time: The CPU picks the job with the shortest remaining execution time.
- Interruptions Allowed: A running job can be paused if a shorter job arrives.
- Dynamic Selection: The CPU constantly checks which job is the shortest.
- Next Job: When the current job finishes, the CPU selects the shortest remaining job.

1. PROCESOS LLEGANDO

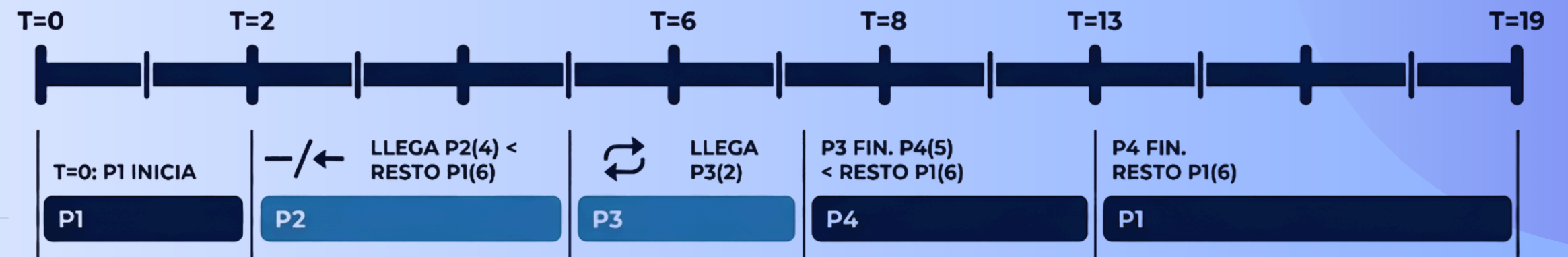
P1 (8)

P2 (4)

P3 (2)

P4 (5)

2. PLANIFICACIÓN Y PREEMPCIÓN



IT CAN BE USEFUL IN SYSTEMS WHERE REDUCING RESPONSE TIME IS CRITICAL AND SHORT PROCESSES NEED TO BE HANDLED QUICKLY.

SJF PREEMPTIVE

```
public void newProcess(boolean cpuEmpty) {
    if (!cpuEmpty) {
        os.interrupt(InterruptType.SCHEDULER_CPU_TO_RQ, null);
    }
}

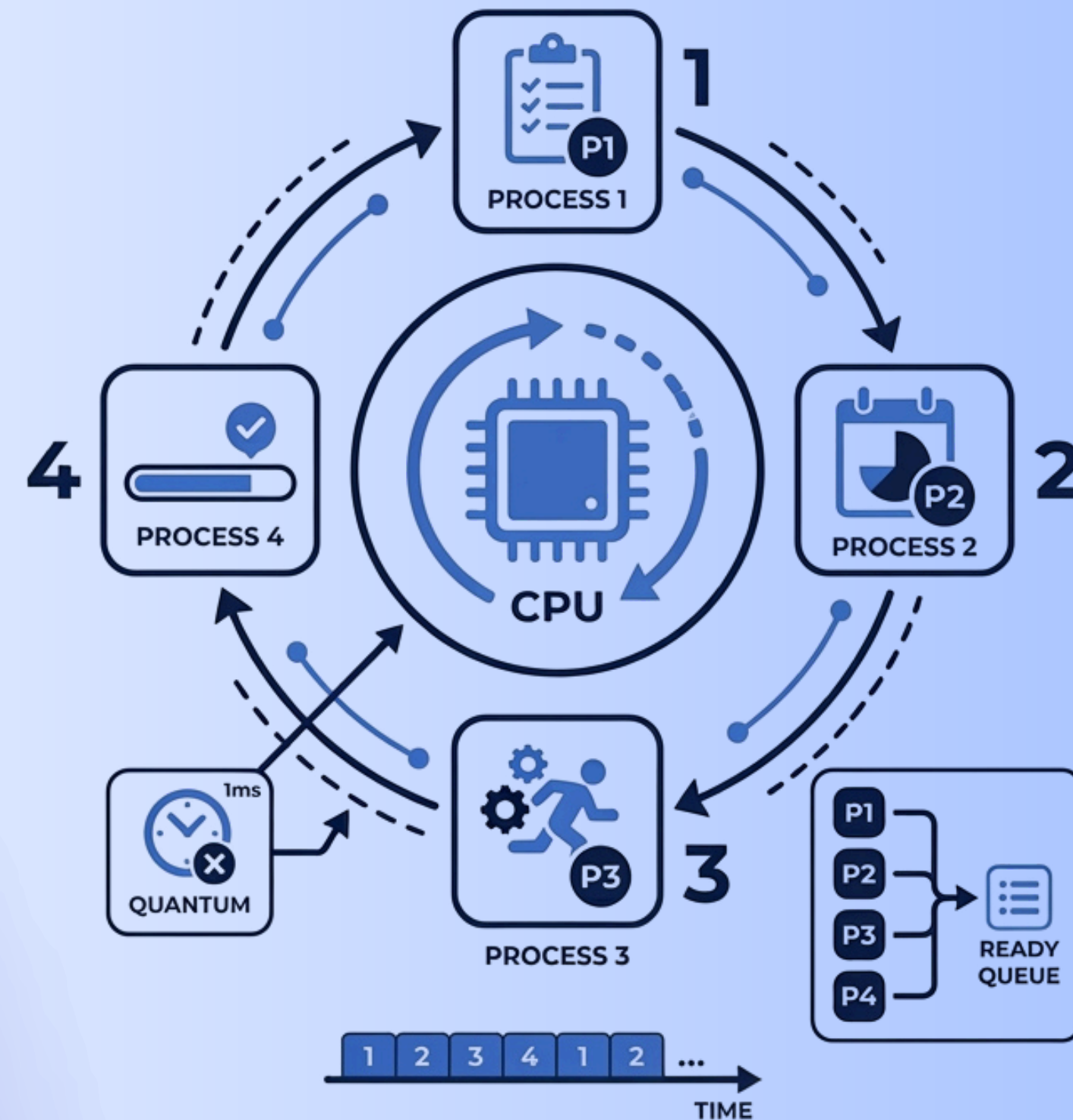
public void IOReturningProcess(boolean cpuEmpty) {
    if (!cpuEmpty) {
        os.interrupt(InterruptType.SCHEDULER_CPU_TO_RQ, null);
    }
}

public void getNext(boolean cpuEmpty) {
    if (!processes.isEmpty() && cpuEmpty) {
        Process shortest = null;

        for (Process p : processes) {
            if (shortest == null) {
                shortest = p;
            } else if (p.getRemainingTimeInCurrentBurst() < shortest.getRemainingTimeInCurrentBurst()) {
                shortest = p;
            } else if (p.getRemainingTimeInCurrentBurst()
                == shortest.getRemainingTimeInCurrentBurst()) {
                shortest = tieBreaker(shortest, p);
            }
        }

        if (shortest != null) {
            processes.remove(shortest);
            os.interrupt(InterruptType.SCHEDULER_RQ_TO_CPU, shortest);
        }
    }
}
```

ROUND ROBIN



- Time Quantum: The CPU gives each job a small, fixed amount of execution time.
- Time Limit: When the time is over, the running job is paused.

- Turn Rotation: The CPU moves to the next job in the queue.
- Repeat the Cycle: Paused jobs return to the queue and wait for their next turn.

ITS PRIMARY APPLICATION IS IN INTERACTIVE SYSTEMS, BECAUSE IT ALLOWS DIFFERENT PROCESSES TO CONTINUOUSLY RECEIVE CPU TIME AND PREVENTS ANY SINGLE PROCESS FROM MONOPOLIZING THE PROCESSOR.

ROUND ROBIN



Universidad del
Rosario

```
public class RoundRobin extends Scheduler{
    int q;
    int cont;
    boolean multiqueue;
    RoundRobin(OS os){
        super(os);
        q = 5;
        cont=0;
    }
    RoundRobin(OS os, int q){
        this(os);
        this.q = q;
    }
    RoundRobin(OS os, int q, boolean multiqueue){
        this(os);
        this.q = q;
        this.multiqueue = multiqueue;
    }
    void resetCounter(){
        cont=0;
    }

    public void getNext(boolean cpuEmpty) {
        if (!cpuEmpty && !processes.isEmpty() && cont >= (q - 1)) {
            Process current = os.getProcessInCPU();
            os.interrupt(InterruptType.SCHEDULER_CPU_TO_RQ, current);

            Process next = processes.remove(0);
            os.interrupt(InterruptType.SCHEDULER_RQ_TO_CPU, next);
            resetCounter();
        }else if (cpuEmpty && !processes.isEmpty()) {
            Process p = processes.remove(0);
            os.interrupt(InterruptType.SCHEDULER_RQ_TO_CPU, p);
            resetCounter();
        } else {
            cont++;
        }
    }
}
```

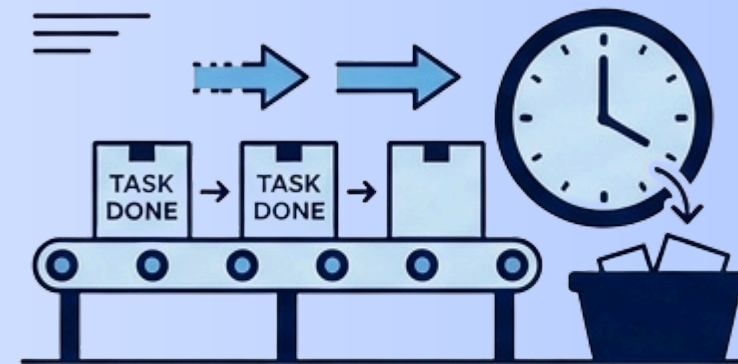

PERFORMANCE EVALUATION INDICATORS

CPU UTILIZATION



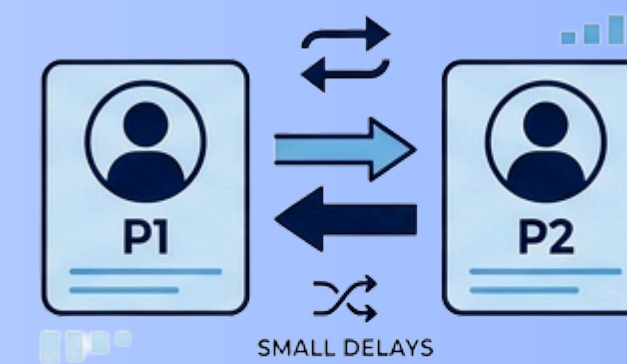
Percentage of time the CPU is busy.

THROUGHPUT



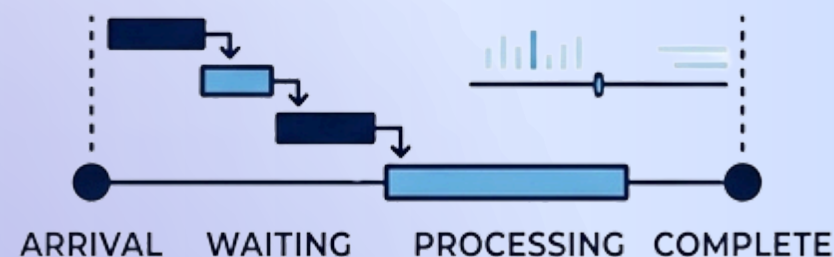
Processes completed per unit of time.

CONTEXT SWITCHES



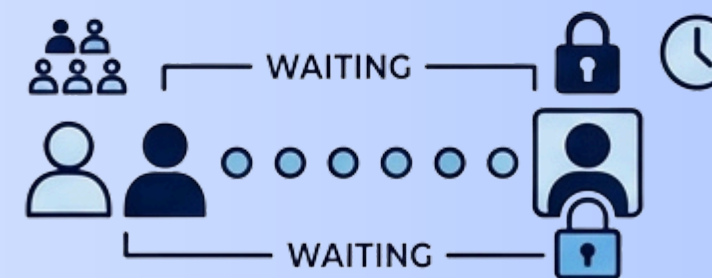
Overhead of changing context between processes.

TURNAROUND TIME



Total time from process arrival to final completion.

WAITING TIME



Time spent waiting in the Ready Queue for CPU access.

RESPONSE TIME



Time from submission to the start of the first execution.

WE COULDN'T SIMPLY SAY THAT AN ALGORITHM WAS BETTER BECAUSE THE SIMULATION LOOKED GOOD. WE NEEDED METRICS.

PERFORMANCE EVALUATION INDICATORS

Expected results:

SJF-NP

Simpler

*****SIMULATION FINISHES*****

*****Process Execution*****

0 0 0 0 0 1 1 1 3 3 3 3 0 0 0 1 1 1 1 1 1 3 3 3 3 3 3 2 2 2 2 2 2 2 -1 -1 -1 2 2 2 2 2

*****Performance Indicators*****

Total execution cycles: 43

CPU Utilization: 0.9302325581395349

Throughput: 0.09302325581395349

Average Turnaround Time: 22.75

Average Waiting Time: 9.0

Average Context Switches (solo Gantt): 2.0

Average Context Switches (completo): 2.0

Average Response Time: 6.25

SJF-P

Simpler

*****SIMULATION FINISHES*****

*****Process Execution*****

0 0 1 1 1 0 0 0 3 3 3 3 0 0 0 1 1 1 1 1 1 3 3 3 3 3 3 2 2 2 2 2 2 2 -1 -1 -1 2 2 2 2 2

*****Performance Indicators*****

Total execution cycles: 43

CPU Utilization: 0.9302325581395349

Throughput: 0.09302325581395349

Average Turnaround Time: 22.75

Average Waiting Time: 9.0

Average Context Switches (solo Gantt): 2.25

Average Context Switches (completo): 2.75

Average Response Time: 5.5

RR-4

Simpler

*****SIMULATION FINISHES*****

*****Process Execution*****

0 0 0 0 1 1 1 0 2 2 2 2 3 3 3 3 1 1 1 1 0 0 0 2 2 2 3 3 3 3 1 1 2 2 2 2 3 3 3 2

*****Performance Indicators*****

Total execution cycles: 40

CPU Utilization: 1.0

Throughput: 0.1

Average Turnaround Time: 29.5

Average Waiting Time: 15.75

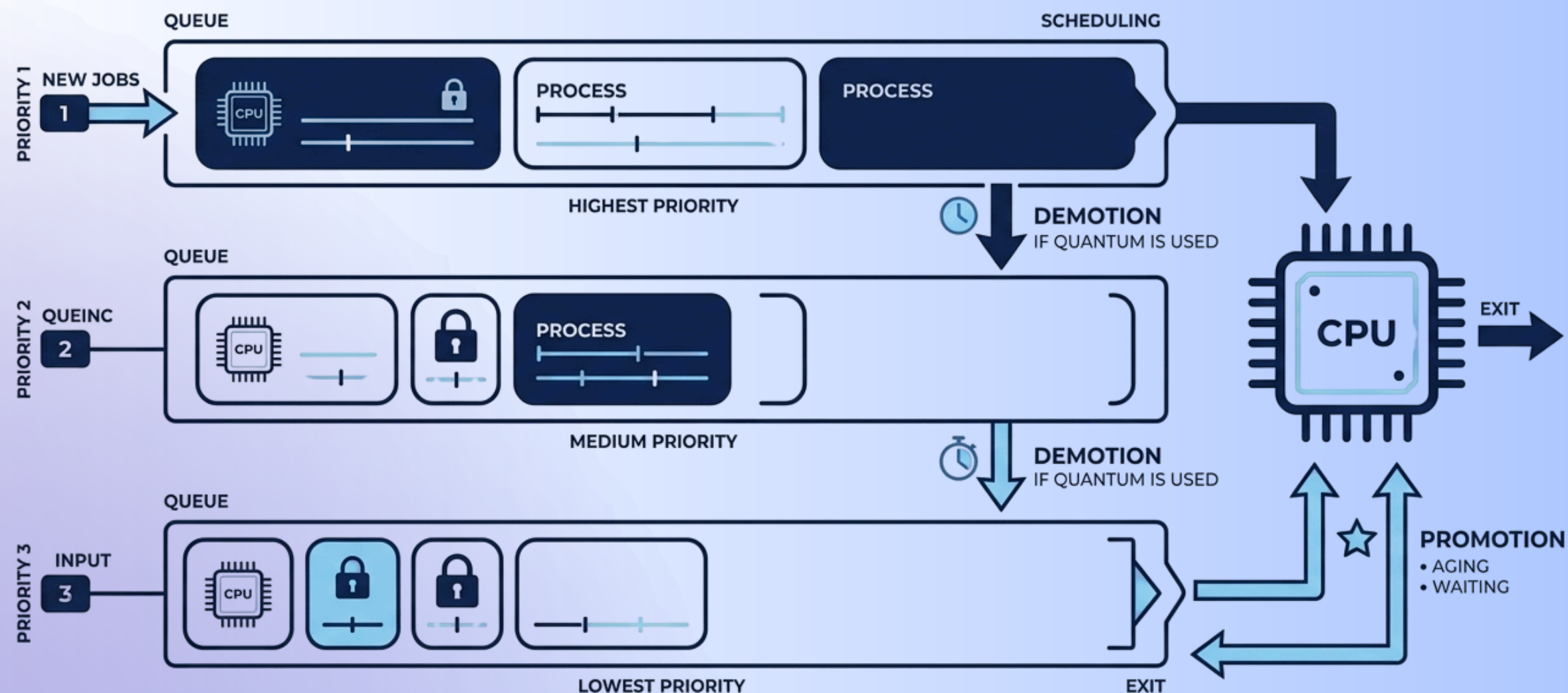
Average Context Switches (solo Gantt): 3.25

Average Context Switches (completo): 3.25

Average Response Time: 2.0

PRIORITY QUEUE

PRIORITY QUEUE (MULTIPLE QUEUES)



- Priority Levels: Processes are divided into different queues according to their priority.
- Highest Priority First: The CPU always chooses a process from the highest-priority queue available.
- Queue Order: If several processes have the same priority, they are handled according to their queue's scheduling method.
- Lower Priority Waits: Lower-priority processes only run when there are no processes waiting in higher-priority queues.

THIS MODEL IS USEFUL WHEN CERTAIN TYPES OF PROCESSES MUST TAKE PRECEDENCE OVER OTHERS—FOR EXAMPLE, CRITICAL PROCESSES VERSUS LOWER-PRIORITY TASKS.

PRIORITY QUEUE

(MAIN LOGIC)

```
public void addProcess(Process p) {
    int prio = p.getPriority();
    schedulers.get(prio).addProcess(p);
}

void defineCurrentScheduler() {
    currentScheduler = -1;

    for (int i = 0; i < schedulers.size(); i++) {
        if (!schedulers.get(i).isEmpty()) {
            currentScheduler = i;
            break;
        }
    }
}

public void getNext(boolean cpuEmpty) {
    if (os.isCPUEmpty()) {
        defineCurrentScheduler();

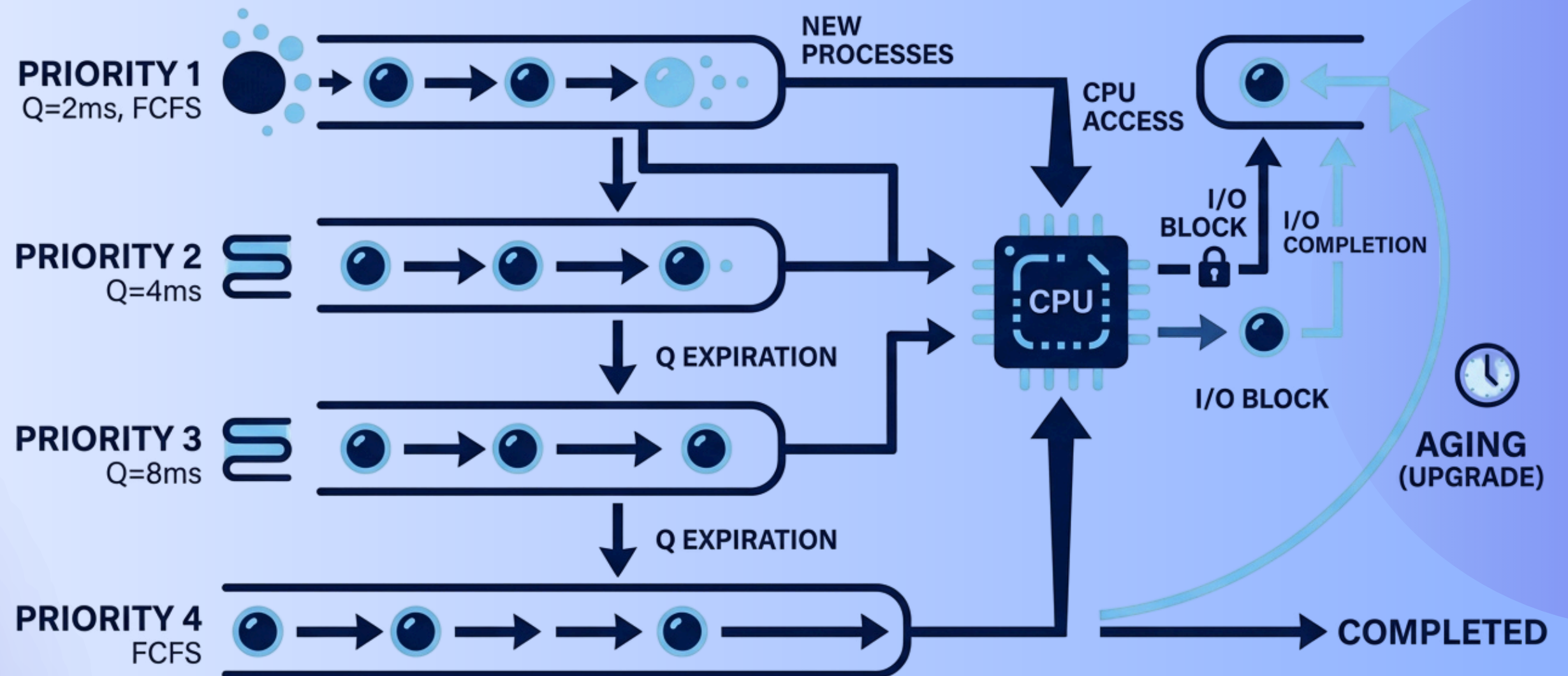
        if (currentScheduler != -1) {
            schedulers.get(currentScheduler).getNext(true);
        }
    }
}
```


MULTI

FEEDBACK QUEUE

- Multiple Queues: Processes are placed in different queues based on their priority.
- Different Time Quantum: Higher-priority queues usually get shorter execution times.
- Process Movement: A process can move to a lower or higher queue depending on its behavior.
- Dynamic Priority: The CPU adjusts which process runs based on the current queue and priority.

MULTI-LEVEL FEEDBACK QUEUE QUEUE (MFQ)



THIS IS PARTICULARLY USEFUL IN INTERACTIVE SYSTEMS, WHERE WE WANT TO FAVOR SHORT OR INTERACTIVE PROCESSES WITHOUT PREVENTING LONG-RUNNING PROCESSES FROM EVENTUALLY RECEIVING CPU TIME.

MFQ

(MAIN LOGIC)

```
public void addProcess(Process p) {
    if (p.getState() == ProcessState.NEW ||
        p.getState() == ProcessState.IO) {
        p.setPriority(0);
    } else if (p.getPriority() < schedulers.size() - 1) {
        p.setPriority(p.getPriority() + 1);
    }

    schedulers.get(p.getPriority()).addProcess(p);
}

void defineCurrentScheduler() {
    currentScheduler = -1;

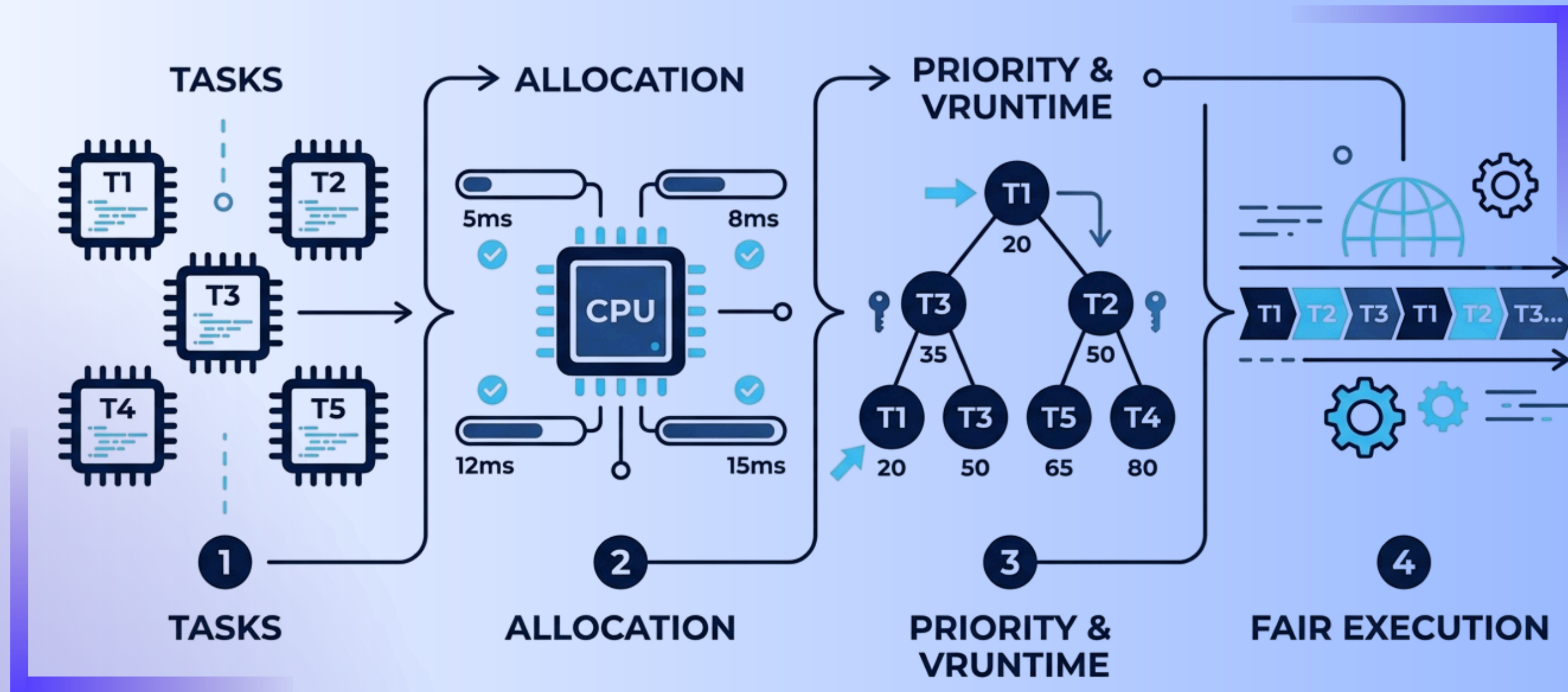
    for (int i = 0; i < schedulers.size(); i++) {
        if (!schedulers.get(i).isEmpty()) {
            currentScheduler = i;
            break;
        }
    }
}

public void getNext(boolean cpuEmpty) {
    if (os.isCPUEmpty()) {
        defineCurrentScheduler();

        if (currentScheduler != -1)
            schedulers.get(currentScheduler).getNext(true);
    }
}
```

FAIR SCHEDULER

- Equal CPU Time: The CPU tries to give each process a similar amount of execution time.
- Least Used First: The CPU selects the process that has used the least CPU time.



- Fair Distribution: Processes take turns so that no process gets all the CPU.
- Balanced Execution: The goal is to keep CPU usage as fair and balanced as possible.

THE IDEA IS TO ACHIEVE A MORE BALANCED DISTRIBUTION OF CPU RESOURCES AMONG PROCESSES—SOMETHING PARTICULARLY USEFUL WHEN WE WANT TO PREVENT A SINGLE PROCESS FROM CONSISTENTLY RECEIVING SIGNIFICANTLY MORE PROCESSING TIME THAN THE OTHERS.


```
private Process pickFairest() {
    Process fairest = null;

    for (Process p : processes) {
        if (fairest == null ||
            getUsedTime(p) < getUsedTime(fairest)) {
            fairest = p;
        }
    }
    return fairest;
}

public void getNext(boolean cpuEmpty) {

    if (cpuEmpty) {
        Process next = pickFairest();
        processes.remove(next);
        os.interrupt(
            InterruptType.SCHEDULER_RQ_TO_CPU, next
        );
        resetCounter();
        return;
    }

    Process current = os.getProcessInCPU();

    if (current != null)
        cpuTimeUsed.merge(
            current.getPid(), 1, Integer::sum
        );

    cont++;

    if (!processes.isEmpty() && cont >= someValue) {
        Process fairest = pickFairest();

        if (current != null &&
            getUsedTime(fairest) < getUsedTime(current)) {

            os.interrupt(
                InterruptType.SCHEDULER_CPU_TO_RQ, current
            );
            processes.remove(fairest);
            os.interrupt(
                InterruptType.SCHEDULER_RQ_TO_CPU, fairest
            );
        }

        resetCounter();
    }
}
```

PERFORMANCE EVALUATION INDICATORS

```
Results for the Priority(simple 2):  
] ***SIMULATION FINISHES***  
***Process Execution***  
0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0  
***Performance Indicators***  
Total execution cycles: 110  
CPU Utilization: 98.181818181819%  
Throughput: 0.036363636363636 procs / cycle  
Average Turnaround Time: 73.0 cycles / process  
Average Waiting Time: 39.25  
Average Context Switches (solo Gantt): 0.0  
Average Context Switches (completo): 3.25  
Average Response Time: 29.75
```

```
For MFQ:  
***SIMULATION FINISHES***  
***Process Execution***  
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 0 0 0 0 0  
***Performance Indicators***  
Total execution cycles: 108  
CPU Utilization: 100.0%  
Throughput: 0.037037037037037 procs / cycle  
Average Turnaround Time: 81.25 cycles / process  
Average Waiting Time: 47.5  
Average Context Switches (solo Gantt): 0.0  
Average Context Switches (completo): 2.0  
Average Response Time: 13.75
```

CONCLUSIONS

ALGORITHM

DECISION RULE

SJF-NP

shortest process

SJF-P

shortest process + preemption

RR

turn by quantum

Priority

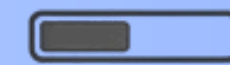
priority level

MFQ

priority + behavior

Fair

accumulated CPU time





THANK YOU!